```
    static long cnt1 = 0;
    static long cnt2 = 0;

    static long fibonacciR(int n) {
        cnt1++;
        if (n == 0 || n == 1) {
            return 1;
        } else {
            return fibonacciR(n - 1) + fibonacciR(n - 2);
        }
    }

    static long fibonacciDP(int n, HashMap<Integer, Long> mem) {
        cnt2++;
        if (n == 0 || n == 1) {
            return 1;
        }
        try {
            return mem.get(n);
        } catch (Exception e) {
            long rslt = fibonacciDP(n - 1, mem) + fibonacciDP(n - 2, mem);
            mem.put(n, rslt);
            return rslt;
        }
    }

    public static void main(String[] args) {
        System.out.println("fib(" + 20 + ")=" + fibonacciR(20));
        System.out.println("----------------------");
        System.out.println("fib(" + 20 + ")=" + fibonacciDP(20, new
HashMap<>()));
        System.out.println(cnt1 + " " + cnt2);
    }
}
```

Κώδικας 18.10 *Η αναδρομική συνάρτηση Fibonacci χωρίς απομνημόνευση και με απομνημόνευση*

Ο κώδικα 18.10 παρουσιάζει δύο εκδόσεις της Fibonacci. Και οι δύο εκδόσεις είναι αναδρομικές. Η συνάρτηση FibonacciR είναι απλή αναδρομική και η FibonacciDP είναι αναδρομική με απομνημόνευση. Ο μετρητής cnt1 μετράει τις κλήσεις στην απλή αναδρομική και ο cnt2 μετράει τις κλήσεις στην αναδρομική με απομνημόνευση. Η έξοδος του κώδικα είναι:

```
fib(20)=10946
----------------------
fib(20)=10946
21891 39
```

Επομένως προκειμένου να υπολογισθεί ο 20ός όρος της ακολουθίας η απλή αναδρομική συνάρτηση πραγματοποιεί 21891 κλήσεις κι η αναδρομική με απομνημόνευση πραγματοποιεί 39 κλήσεις.

Η FibonacciDP αξιοποιεί ένα HashMap<Integer, Long>. Ο Integer σε αυτό το HashMap αντιπροσωπεύει την τάξη του όρου της ακολουθίας και ο Long την τιμή του αντίστοιχου όρου. Κατά την κλήση της συνάρτησης, γίνεται προσπάθεια να ανακτηθεί από το HashMap η τιμή του ζητούμενου όρου. Εφόσον αυτή δεν βρεθεί, παράγεται εξαίρεση. Κατά τη σύλληψη της εξαίρεσης υπολογίζεται ο όρος και εισάγεται στο HashMap, έτσι την επόμενη φορά που θα ζητηθεί ο ίδιος όρος θα βρεθεί στο HashMap και δεν θα απαιτηθεί ο επανυπολογισμός του.

18.3 Λυμένες ασκήσεις

18.3.1 Εφαρμογή εκφράσεων λάμδα

Δημιουργήστε τρεις εκφράσεις λάμδα. Η πρώτη θα υπολογίζει το άθροισμα των στοιχείων ενός πίνακα πραγματικών, η δεύτερη θα υπολογίζει τον μέσο όρο των στοιχείων ενός πίνακα πραγματικών και η τρίτη το ελάχιστο στοιχείο ενός πίνακα πραγματικών. Εκχωρήστε τις εκφράσεις λάμδα σε μεταβλητές ονομαζόμενες addition, average, minElement, αντίστοιχα. Προσθέστε main στην οποία υπολογίστε το άθροισμα, τον μέσο όρο και το ελάχιστο στοιχείο ενός πίνακα πραγματικών χρησιμοποιώντας τις αντίστοιχες μεταβλητές.

Λύση

```
interface ArrayProcessor {
    double apply( double[] array );
}

public class Processor {

static ArrayProcessor addition= array-> {
    double sum=0;
    for (double d: array)
        sum+=d;
    return sum;
};

static ArrayProcessor average=array-> {
    return addition.apply(array)/array.length;
};

static ArrayProcessor minElement=array-> {
    double min=array[0];
    for (int i=1; i<array.length; i++)
        if (array[i]<min) {
            min=array[i];
        }
    return min;
};

    public static void main(String[] args) {
        double[] array=new double[]{1,2,3,4};
        double sum=addition.apply(array);
        System.out.println(sum);
        System.out.println(average.apply(array));
        System.out.println(minElement.apply(array));
    }

}
```

Κώδικας 18.11 *Υπολογισμοί σε πίνακα πραγματικών με εκφράσεις λάμδα*

Καταρχάς, προκειμένου να ορίσουμε τις εκφράσεις λάμδα, θα πρέπει να έχουμε στη διάθεσή μας κατάλληλη λειτουργική διεπαφή, δηλαδή μία διεπαφή που δηλώνει μία αφηρημένη μέθοδο η οποία λαμβάνει ως παράμετρο έναν πίνακα πραγματικών και επιστρέφει έναν πραγματικό αριθμό. Πράγματι, η διεπαφή ArrayProcessor που δηλώνεται στην αρχή του κώδικα 18.11 πληροί τις απαιτούμενες προδιαγραφές.

Στη συνέχεια, ορίζουμε τις εκφράσεις λάμδα τις οποίες εκχωρούμε στις αντίστοιχες μεταβλητές των οποίων ο τύπος είναι ArrayProcessor. Στην main καλούμε διαμέσου των μεταβλητών την apply δίνοντάς της έναν πίνακα πραγματικών ως παράμετρο.

18.3.2 Αναδρομικός υπολογισμός δύναμης με απομνημόνευση

Υλοποιήστε μια συνάρτηση αναδρομικού υπολογισμού δύναμης με πραγματική βάση υψωμένη σε ακέραιο εκθέτη χωρίς απομνημόνευση και μία με απομνημόνευση. Μελετήστε πώς ακριβώς θα πρέπει να αναπτυχθεί η συνάρτηση με απομνημόνευση ώστε να έχουμε περιορισμό των αναδρομικών κλήσεων.

Λύση

```
class Tupple {

    double b;
    int e;

    public Tupple(double b, int e) {
        this.b = b;
        this.e = e;
    }

    @Override
    public int hashCode() {
        int hash = 3;
        hash = 43 * hash + (int) this.b;
        hash = 43 * hash + this.e;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Tupple other = (Tupple) obj;
        if (this.b != other.b) {
            return false;
        }
        return (int) this.e == (int) other.e;
    }

}

public class Power {

    static long cnt1 = 0;
    static long cnt2 = 0;
    static HashMap<Tupple, Double> mem = new HashMap<>();

    static double pow(int b, int e) {
        cnt1++;
        if (e > 0) {
            return b * pow(b, e - 1);
        } else if (e < 0) {
            return 1 / pow(b, -e);
        }
        return 1;
    }

    static double fastPow(int b, int e, HashMap<Tupple, Double> mem) {
        cnt2++;
        if (e != 0) {
            try {
                return mem.get(new Tupple(b, e));
```

```
                } catch (Exception x) {
                    double rslt;
                    if (e > 0) {
                        rslt = b * fastPow(b, e - 1, mem);
                    } else {
                        rslt = 1 / fastPow(b, -e, mem);
                    }
                    mem.put(new Tupple(b, e), rslt);
                    return rslt;
                }
            }
            return 1;
        }

        static void test() {
            cnt1=0;
            cnt2=0;
            System.out.println("pow(2,16)=" + pow(2, 16));
            System.out.println("fastpow(2,16)=" + fastPow(2, 16, mem));
            System.out.println(cnt1 + " " + cnt2);
        }

        public static void main(String[] args) {
            test();
            System.out.println("----------");
            test();
        }
    }
```

Κώδικας 18.12 *Υπολογισμός δύναμης χωρίς και με απομνημόνευση*

Η έξοδος του κώδικα 18.2 έχει ως εξής;

```
pow(2,16)=65536.0
fastpow(2,16)=65536.0
17 17
----------
pow(2,16)=65536.0
fastpow(2,16)=65536.0
17 1
```

Αυτό σημαίνει πως κατά την πρώτη κλήση της test και οι δύο εκδόσεις του υπολογισμού της δύναμης πραγματοποίησαν από 17 αναδρομικές κλήσεις. Ωστόσο, κατά τη δεύτερη κλήση της test, η έκδοση χωρίς απομνημόνευση πραγματοποίησε και πάλι 17 αναδρομικές κλήσεις, ενώ η έκδοση με απομνημόνευση μόνο την αρχική κλήση στην fastPow. Πράγματι κατά τον αναδρομικό υπολογισμό της δύναμης δεν επαναλαμβάνεται κλήση με ίδιες παραμέτρους. Αν για παράδειγμα θελήσουμε να υπολογίσουμε το 2^3, θα πρέπει να υπολογιστούν οι δυνάμεις 2^2, 2^1 και 2^0. Επομένως, κανένας υπολογισμός δεν επαναλαμβάνεται. Ωστόσο, αν έχουμε μία συνάρτηση υπολογισμού δύναμης, είναι σχεδόν βέβαιο ότι θα την χρησιμοποιήσουμε επαναληπτικά σε μία εφαρμογή. Τότε προκύπτει το όφελος. Όμως αυτό το όφελος δεν θα προκύψει αν κατά την κλήση της συνάρτησης περάσουμε μία νέα απεικόνιση ως παράμετρο, όπως κάναμε με τη συνάρτηση Fibonacci. Αντίθετα, χρειαζόμαστε μια στατική απεικόνιση στην ίδια κλάση που περιλαμβάνει την αναδρομική συνάρτηση ώστε τα αποτελέσματα των υπολογισμών να είναι δεδομένα μεταξύ διαφορετικών κλήσεων.

Βιβλιογραφία

[1] S. G. Ganesh, H. K. Kumar, and T. Sharma, Oracle Certified Professional Java SE 8 Programmer Exam 1Z0-809: A Comprehensive OCPJP 8 Certification Guide, 2nd ed. USA: Apress, 2015.

[2] “Lambda Expressions (The Java™ Tutorials > Learning the Java Language > Classes and Objects).” https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html#syntax (accessed Jan. 04, 2022).

[3] “Method References (The Java™ Tutorials > Learning the Java Language > Classes and Objects).” https://docs.oracle.com/javase/tutorial/java/javaOO/methodreferences.html (accessed Jan. 05, 2022).

Πρόχειρο αντίτυπο υπο έκδοση εγχειριδίου από τις εκδόσεις Κάλλιπος

Κριτήρια Αξιολόγησης

Αμεταβλητότητα

Έστω ο κώδικας

```
public class KA01 {

	public static void main(String args[]) {
		String t = "12345";
		t.replace("2", "A");
		System.out.println(t);
	}
}
```

Ποια είναι η έξοδός του;

```
A. 12345
B. 1A345
Γ. 12A45
```

Απάντηση/Λύση

Όπως έχουμε εξηγήσει, τα Strings είναι αμετάβλητα. Από τη στιγμή που θα δημιουργηθούν, διατηρούν την τιμή τους μέχρι το πέρας της εφαρμογής. Οι συναρτήσεις της κλάσης String, όπως η replace δεν μεταβάλλουν το String από το οποίο καλούνται, αλλά δημιουργούν και επιστρέφουν ένα νέο String. Το String που επέστρεψε η replace δεν το αξιοποιεί ο κώδικας της ερώτησης. Η δε αναφορά s συνεχίζει να δείχνει στο String “12345”. Έτσι, η σωστή απάντηση είναι η Α.

Αρχικοποίηση

Έστω ο κώδικας

```
public class KA02 {

	static double x;

	public static double calc(double value) {
		int a, b, c;
		if (x < 10) {
			a = 2;
			b = 3;
			c = 4;
		}
		return a * b * c * value;
	}

	public static void main(String[] args) {
		System.out.println(calc(100));
	}
}
```

Ποια είναι η έξοδός του;

```
A. 240
B. 2400
```

```
Γ. Λάθος μεταγλώττισης
Δ. 0
```

Απάντηση/Λύση

Η στατική μεταβλητή x ως μέλος της κλάσης αρχικοποιείται αυτόματα στο 0. Επομένως, αν εκτελεστεί η if μέσα στην calc, η έκφραση x<10 θα είναι αληθής και άρα ο κώδικας θα εισέλθει μέσα στο μπλοκ της if και θα αρχικοποιήσει τις a, b, c. Ωστόσο, αυτό είναι κάτι που δεν μπορεί να υπολογιστεί κατά τον χρόνο μεταγλώττισης, αλλά μόνο κατά τον χρόνο εκτέλεσης. Παράλληλα, οι τοπικές μεταβλητές σε αντίθεση με τα μέλη της κλάσης δεν αρχικοποιούνται αυτόματα. Επομένως, για τον μεταγλωττιστή, στην return της calc, οι μεταβλητές a, b, c ενδέχεται να είναι αναρχικοποίητες, αποκρίνεται με λάθος μεταγλώττισης και η σωστή απάντηση είναι η Γ.

Η main

Τι θα συμβεί αν επιχειρήσουμε να μεταγλωττίσουμε τον ακόλουθο κωδικα;

```
public class KA03 {

	public static int main(String[] args) {
		System.out.println("Hello World!");
		return 0;
	}
}

A. Λάθος μεταγλώττισης
B. Θα μεταγλωττιστεί και θα τυπώσει Hello World!
Γ. Θα μεταγλωττιστεί και θα τυπώσει 0
Δ. Θα μεταγλωττιστεί αλλά δεν θα τυπώσει τίποτα.
```

Απάντηση/Λύση

Ο κώδικας δεν έχει κάποιο θέμα με τη μεταγλώττιση, οπότε η επιλογή Α δεν ισχύει. Επομένως, ο κώδικας θα μεταγλωττιστεί. Κατά την εκτέλεση θα αναζητηθεί η main. Ο κώδικας περιλαμβάνει μία main, ωστόσο είναι διαφορετική από την main που λειτουργεί ως κύρια είσοδος σε εφαρμογή Java. Η μεν main του κώδικα επιστρέφει int, η δε main που αναγνωρίζεται ως κύρια είσοδος στο πρόγραμμα είναι τύπου void. Επομένως, ο κώδικας δεν θα τρέξει, άρα αποκλείονται η Β και η Γ και η σωστή απάντηση είναι η Δ. Αν επιχειρήσουμε να τρέξουμε τον κώδικα, η JVM θα αποκριθεί με μήνυμα πως δεν βρήκε main.

Μεταβλητές και Συναρτήσεις στιγμιότυπου

Ο κώδικας

```
class Alpha {

	public int h = 1;

	public int getH() {
		return h;
	}
}

public class KA04 extends Alpha {

	public int h = 11;

	@Override
```

```
        public int getH() {
              return h;
        }

        public static void main(String[] args) {
              Alpha parent = new KA04();
              System.out.print(parent.h + "-" + parent.getH() + "-
              >");
              KA04 child = (KA04) parent;
              System.out.println(child.h + "-" + child.getH());
        }
}
```

θα εμφανίσει

```
A. 1-11->11-11
B. 11-11->11-11
Γ. 1-11->1-11
Δ. Τίποτε από τα παραπάνω
```

Απάντηση/Λύση

Εδώ έχουμε δύο αναφορές, την parent και την child. Ωστόσο και οι δύο αναφορές δείχνουν στο ίδιο πραγματικό αντικείμενο, αυτό που δημιουργείται στην πρώτη γραμμή της main. Άσχετα από τον τύπο της αναφοράς, η συνάρτηση getH που θα τρέξει είναι η συνάρτηση του πραγματικού και όχι του τυπικού αντικειμένου. Επομένως, η συνάρτηση θα εμφανίσει και τις 2 φορές που καλείται την τιμή 11. Δεν ισχύει το ίδιο όμως με τη μεταβλητή. Η μεταβλητή θα επιλυθεί με βάση τον τύπο του αντικειμένου και όχι με βάση το πραγματικό αντικείμενο. Έτσι η σωστή απάντηση είναι η Α. Αν ωστόσο στην κλάση KA04 δεν είχε οριστεί μεταβλητή h, τότε σωστή θα ήταν η απάντηση Γ.

Αξιολόγηση βραχέως κυκλώματος

Τι θα εμφανίσει ο ακόλουθος κώδικας κατά την εκτέλεσή του;

```
class KA05 {

      static boolean a;
      static boolean b;
      static boolean c;

      public static void main(String[] args) {
            boolean bool = (a = true) || (b = true) && (c =
            true);
            System.out.println(a + ", " + b + ", " + c);
      }
}

A. true, true, true
B. true, false, false
Γ. true, false, true
Δ. false, false, false
Ε. Λάθος μεταγλώττισης
```

Απάντηση/Λύση

Τα στατικά μέλη a, b, c αρχικοποιούνται αυτόματα στην τιμή false. Γενικότερα, ο κώδικας δεν περιέχει κάποιο λάθος μεταγλώττισης, οπότε η απάντηση Ε είναι λάθος. Στην main η παράσταση που εκχωρείται στην bool

αξιολογείται από αριστερά προς τα δεξιά. Προσέξτε πως η παράσταση αυτή είναι μία σειρά από εκχωρήσεις και όχι λογικές συγκρίσεις που λειτουργούν ως τελεστές λογικών πράξεων. Στη μεταβλητή a επομένως εκχωρείται η τιμή true και το αποτέλεσμα της (a = true) είναι true. Εφόσον η τιμή true συνδέεται με τον τελεστή ||, γνωρίζουμε πως λόγω της αξιολόγησης βραχέως κυκλώματος, το υπόλοιπο τμήμα της παράστασης δεν θα αξιολογηθεί. Συνεπώς, στις μεταβλητές b και c δεν εκχωρείται άλλη τιμή. Αυτές διατηρούν την αρχική τιμή τους, δηλαδή την τιμή false. Επομένως, σωστή απάντηση είναι η Β.

Μεταβλητές τιμής

Ο κώδικας

```
public class KA06 {

     int k = 5;

     public boolean checkIt(int k) {
          return k-- > 0;
     }

     public void printThem() {
          while (checkIt(k)) {
               System.out.print(k);
          }
     }

     public static void main(String[] args) {
          new KA06().printThem();
     }
}
```

εμφανίζει

```
Α. 54321
Β. 43210
Γ. Λάθος μεταγλώττισης
Δ. Λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Η printThem καλείται στην main. Αυτή σε μια επαναληπτική διαδικασία που ελέγχεται από την checkIt(k) τυπώνει την τιμή του k. Μέσα στην checkIt μειώνεται η τιμή του k κατά 1. Ωστόσο, το k μέσα στην checkIt δεν είναι το ίδιο με το k μεταβλητή στιγμιότυπου της κλάσης. Το k μέσα στην checkIt είναι παράμετρος τιμής και λειτουργεί ως τοπική μεταβλητή αφήνοντας αμετάβλητη την τιμή της μεταβλητής του στιγμιότυπου. Επομένως, η checkIt με κάθε κλήση της τυπώνει το 5 και το πρόγραμμα πέφτει σε ατέρμονη επαναληπτική διαδικασία, άρα η σωστή απάντηση είναι η Δ.

Δισδιάστατοι πίνακες

Ποια είναι η έξοδος της main;

```
public class KA07 {

     public static void main(String[] args) {
          int[][] t = {{1, 2}, {3, 4}, {5, 6}};
          for (int i = 0; i < t.length; i++) {
               for (int j = 0; j < t.length; j++) {
```

```
                        System.out.print(t[i][j]);
                    }
                }
            }
        }
```

A. 12345
B. Λάθος μεταγλώττισης
Γ. Λάθος χρόνου εκτέλεσης

Απάντηση/Λύση

Όπως έχουμε εξηγήσει, οι δισδιάστατοι πίνακες είναι στην ουσία πίνακες που το κάθε στοιχείο τους είναι ένας μονοδιάστατος πίνακας. Ο πίνακας t περιέχει τους πίνακες {1, 2}, {3, 4} και {5, 6}. Οι πίνακες αυτοί έχουν μήκος δύο. Από την άλλη πλευρά, ο ίδιος ο πίνακας t έχει μήκος 3. Στον εμφωλευμένο βρόχο όμως χρησιμοποιούμε το μήκος του t και στις δύο for. Επομένως, ο κώδικας θα επιχειρήσει να προσπελάσει καταρχάς το στοιχείο t[0][2], δηλαδή το στοιχείο στη θέση 2 του πίνακα {1, 2}. Τέτοιο στοιχείο όμως δεν υπάρχει, καθώς η τελευταία θέση του πίνακα είναι η 1. Το πρόγραμμα θα πέσει με ArrayIndexOutOfBoundsException και η σωστή απάντηση είναι η Γ.

Ο προκαθορισμένος Δημιουργός

Ποια είναι η έξοδος της main;

```
class A {
    int i;
    public void A() {i=1;}
}

public class KA08 {
    public static void main(String[] args) {
        A a1=new A();
        System.out.println(a1.i);
    }
}
```

A. 0
B. 1
Γ. Λάθος μεταγλώττισης
Δ. Λάθος εκτέλεσης

Απάντηση/Λύση

Καταρχάς δεν υπάρχει λάθος μεταγλώττισης ούτε και εκτέλεσης. Στην main καλείται ο προκαθορισμένος δημιουργός της κλάσης A και δημιουργείται το στιγμιότυπο a1. Προσέξτε όμως πως η A() που ορίζεται στην κλάση A δεν είναι δημιουργός, καθώς έχει χαρακτηριστεί ως void αλλά μία συνάρτηση στιγμιότυπου. Επομένως κατά τη δημιουργία του a1 δεν τρέχει η συνάρτηση A(), αλλά ο προκαθορισμένος δημιουργός. Αυτός βέβαια δεν μεταβάλλει την τιμή της i η οποία έχει αρχικοποιηθεί αυτόματα στην τιμή 0. Έτσι ο κώδικας εμφανίζει 0 και σωστή απάντηση είναι η Α.

Ψηφιολέξεις

Ποια η έξοδος της main;

```
public class KA09 {

    public static void main(String[] args) {
        int i=Integer.MAX_VALUE;
```

```
        int j=i+1;
        int k=~i;
        System.out.print(j==k);
        System.out.println(j==Integer.MIN_VALUE);
    }
}

A. truefalse
B. falsetrue
Γ. truetrue
Δ. falsefalse
```

Απάντηση/Λύση

Ο ακέραιος στην Java είναι 32 bit. Για ευκολία όμως θα υποθέσουμε πως ο ακέραιος στην Java είναι 8 bit. Μια τέτοια υπόθεση δεν επηρεάζει καθόλου το σκεπτικό της απάντησης.

Ποια είναι λοιπόν η αναπαράσταση της μέγιστης τιμής ενός προσημασμένου ακέραιου μήκους 8 bits; Καταρχάς, εφόσον πρόκειται για προσημασμένο ακέραιο, γνωρίζουμε ότι το πρώτο bit αναπαριστά το πρόσημο. Εφόσον μας ενδιαφέρει η μέγιστη τιμή, ο αριθμός είναι θετικός και το πρώτο bit είναι 0. Για να πάρουμε λοιπόν τη μέγιστη τιμή, όλα τα υπόλοιπα bits θα πρέπει να λάβουν τη μέγιστη δυνατή τιμή, δηλαδή την τιμή 1. Άρα ο μέγιστος ακέραιος στα 8 bits είναι ο 01111111. Σημειώστε πως στην πράξη ο ψηφιοχάρτης της μεταβλητής i είναι 0 ακολουθούμενο από 31 άσους.

Η σταθερά 1 έχει προφανώς ως ψηφιοχάρτη την τιμή 00000001. Επομένως, στο j εκχωρείται το άθροισμα 01111111+00000001 που μας δίνει 10000000. Εφόσον το πρώτο bit του αθροίσματος είναι 1 πρόκειται για αρνητικό αριθμό. Εφόσον το μέτρο του είναι 0000000 πρόκειται προφανώς για τον μικρότερο αριθμό που είναι δυνατόν να αναπαρασταθεί με 7 bits, δηλαδή στην i έχει εκχωρηθεί ο τιμή Integer.MIN_VALUE που είναι ίση με 10000000. Στην k εκχωρείται η ίδια τιμή ως συμπλήρωμα του i. Επομένως, η έξοδος του κώδικα είναι truetrue και σωστή είναι η απάντηση Γ.

Πράξεις με πραγματικούς

Ποια η έξοδος της main;

```
public class KA10 {

    public static void main(String[] args) {
        System.out.println((Double.MIN_VALUE < 0) + " " +
(Double.NEGATIVE_INFINITY + Double.POSITIVE_INFINITY) + " " +
(258. / 0));
    }
}

A. true 0 Infinity
B. true NaN Infinity
Γ. false 0 Infinity
Δ. false NaN Infinity
Ε. Λάθος χρόνου εκτέλεσης, διαίρεση με το 0
```

Απάντηση/Λύση

Η διαίρεση με το 0 στην έκφραση 258. / 0 δεν παράγει λάθος χρόνου εκτέλεσης. Καθώς ο ένας από τους δύο τελεστέους είναι τύπου double (ο 258.), η διαίρεση είναι διαίρεση πραγματικών. Μάλιστα καθώς το 258. είναι θετικός το αποτέλεσμα είναι +Infinity. Ο τελεστής + όμως παραλείπεται και έτσι η έκφραση 258./0 αξιολογείται σε infinity. Η Double.MIN_VALUE εκφράζει την ελάχιστη θετική τιμή που μπορεί να αναπαρασταθεί από τον τύπο Double. Επομένως η έκφραση Double.MIN_VALUE < 0 επιστρέφει false. Τέλος, η πρόσθεση Double.NEGATIVE_INFINITY + Double.POSITIVE_INFINITY επιστρέφει NaN και η σωστή απάντηση είναι η Δ.

Σύλληψη εξαιρέσεων

Ποια η έξοδος της main;

```
class MyException extends Exception {
}

class MyException1 extends MyException {
}

public class KA11 {

    void myMethod() throws MyException {
        throw new MyException();
    }


    public static void main(String[] args) throws Exception {
        KA11 et = new KA11();
        try {
            et.myMethod();
        } catch (MyException me) {
            System.out.print("MyException");
        } catch (MyException1 me) {
            System.out.print ("MyException1");
        } finally {
            System.out.print (" finally");
        }
    }
}

A. MyException
B. MyException1
Γ. MyExceptionfinally
Δ. MyException1finally
Ε. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Στο catch συλλαμβάνουμε πρώτα εξαίρεση τύπου MyException και μετά εξαίρεση τύπου MyException1. Ωστόσο, η MyException1 είναι παράγωγη της MyException. Με άλλα λόγια η MyException1 είναι MyException. Έτσι είτε παραχθεί MyException είτε παραχθεί MyException1 θα συλληφθεί από το πρώτο catch. Αυτό σημαίνει ότι ο κώδικας στο δεύτερο catch δεν εκτελείται ποτέ, πράγμα που εντοπίζει ο μεταγλωττιστής και παράγει κατάλληλο μήνυμα. Η σωστή απάντηση είναι η Ε.

Ο κώδικας μπορεί να περάσει με επιτυχία μεταγλώττιση αν αφαιρεθεί το δεύτερο catch. Σε αυτήν την περίπτωση θα έχουμε ενιαία αντιμετώπιση των δύο τύπων εξαιρέσεων. Αν ωστόσο χρειαζόμαστε να διαφοροποιήσουμε μεταξύ MyException και MyException1, τότε θα πρέπει να ελέγξουμε πρώτα την MyException1 και στη συνέχεια την MyException.

instanceof

Ποια η έξοδος της main;

```
class A1{};
class B1 extends A1 {}
class C1 extends B1 {}
```

```
public class KA12 {
    public static void main(String[] args) {
        A1 o = new C1();
        System.out.print(o instanceof A1);
        System.out.print(o instanceof B1);
        System.out.print(o instanceof C1);
    }
}

A. truefalsetrue
B. falsefalsetrue
Γ. truefalsefalse
Δ. truetruetrue
```

Απάντηση/Λύση

Ο τελεστής instanceof ελέγχει το πραγματικό αντικείμενο. Στην αναφορά έχει εκχωρηθεί αντικείμενο τύπου C1. Ένα τέτοιο αντικείμενο είναι ένα C1, επίσης είναι ένα B1 και είναι ένα A1. Επομένως, ο τελεστής θα επιστρέψει true και στους τρεις ελέγχους και η σωστή απάντηση είναι η Δ.

Υπερφόρτωση συναρτήσεων

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA13 {

    static void m(int a) { System.out.print("int"); }
    static void m(char a) {System.out.print("char"); }

    public static void main(String[] args) {
        int i = 'a';
        char k = 127;
        m(i);
        m(k);
    }
}

A. Λάθος μεταγλώττισης
B. Λάθος εκτέλεσης
Γ. intchar
Δ. charint
```

Απάντηση/Λύση

Καταρχάς δεν υπάρχει ούτε λάθος μεταγλώττισης ούτε εκτέλεσης. Στη μεταβλητή i εκχωρείται η σταθερά ‘a’. Η σταθερά είναι τύπου χαρακτήρα. Οι χαρακτήρες όπως έχουμε εξηγήσει είναι ακέραιοι τύποι. Εκείνο που αποθηκεύεται είναι ο ακέραιος κωδικός του χαρακτήρα. Κατά την εκχώρηση i = ‘a’ λοιπόν στο i εκχωρείται μία ακέραια τιμή. Άσχετα από το ότι η τιμή αυτή προήλθε από έναν χαρακτήρα όταν προσπελαύνεται μέσω μιας ακέραιας μεταβλητής ερμηνεύεται σαν ακέραιος. Αντίστοιχα λειτουργεί και η εκχώρηση k = 127. Άσχετα με το ότι στην k εκχωρήσαμε μια ακέραιη τιμή, η k παραμένει τύπου χαρακτήρα. Επομένως, η επίλυση των κλήσεων της m στην main είναι σαφής. Στην κλήση m(i) καλείται η m με παράμετρο int και στην κλήση m(k) καλείται η m με παράμετρο char. Ο κώδικας εμφανίζει intchar και σωστή είναι η απάντηση Γ.

Σειρά αξιολόγησης

Ποια η έξοδος του κώδικα που ακολουθεί;

```
public class KA14 {
    public static void main(String[] args) {
        String s="John";
        String s1=new String(s);
        System.out.print((s==s1)+" "+s.equals(s1));
        System.out.print(" ");
        System.out.print(s==s1+" "+s.equals(s1));
    }
}

A. false true false true
B. false true true false
Γ. false false false
Δ. false true false
```

Απάντηση/Λύση

Η πρώτη print εμφανίζει false true. Η έκφραση (s==s1) είναι false, καθώς με τον τελεστή ισότητας συγκρίνονται οι αναφορές s και s1 που είναι διαφορετικές. Στη συνέχεια, η έκφραση s.equals(s1) είναι true καθώς s και s1 δείχνουν σε διαφορετικά μεν Strings που έχουν όμως και τα δύο το “John” ως περιεχόμενο.

Στη δεύτερη print όμως τα πράγματα είναι διαφορετικά. Εδώ η έκφραση s==s1 δεν περικλείεται σε παρενθέσεις, άρα δεν έχει προτεραιότητα. Ο τελεστής + έχει υψηλότερη προτεραιότητα από τον τελεστή ==. Επομένως, για να υπολογιστεί η έκφραση s==s1+” “+s.equals(s1) θα πρέπει πρώτα να υπολογιστεί η έκφραση s1+” “+s.equals(s1). Η έκφραση αυτή είναι τύπου String και η τιμή της είναι John true. Στη συνέχεια συγκρίνεται η αναφορά s με την ανώνυμη αναφορά στο String John true, οπότε η print εμφανίζει false. Συνεπώς σωστή είναι η απάντηση Γ.

Παραγωγή εξαίρεσης στο finally μπλοκ

Τι μήνυμα θα λάβουμε στη στοίβα κλήσης κατά την εκτέλεση του κώδικα που ακολουθεί;

```
class RTException extends RuntimeException {
    RTException(String msg) {
        super();
    }
}
public class KA15 {

    static void f() throws Exception {
        try {
            throw new RTException("Inside try");
        } catch (RuntimeException e) {
            throw new Exception("Inside catch");
        } finally {
            throw new RuntimeException("Inside finally");
        }
    }

    public static void main(String args[]) throws Exception {
        f();
    }
}

A. Inside try
```